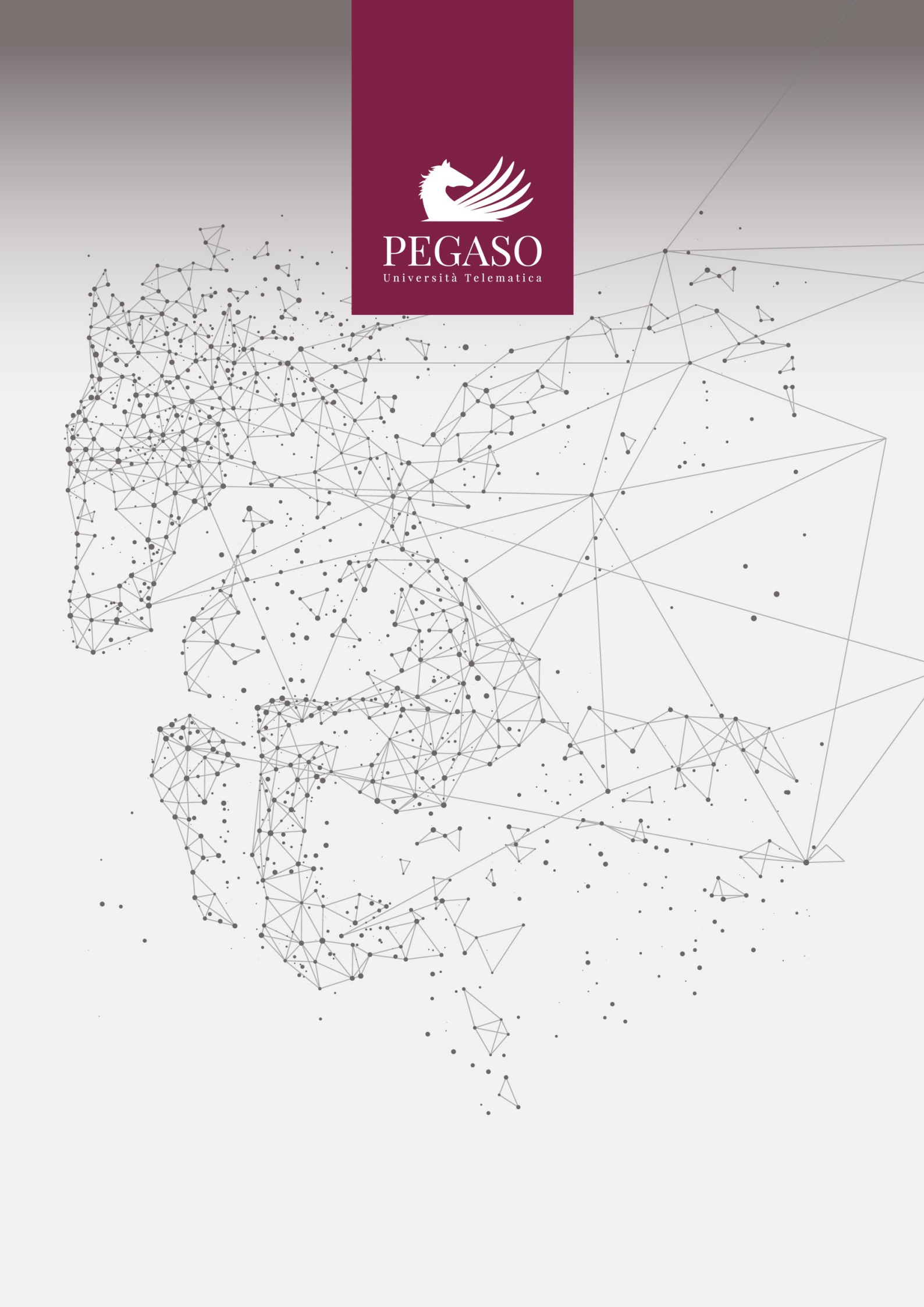

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. AXIOS 3
2. VUEJS 7
3. AXIOS & VUEJS 11
BIBLIOGRAFIA 13
SITOGRAFIA 14

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Axios

Axios è una libreria JavaScript che semplifica notevolmente il processo di effettuare richieste HTTP asincrone verso un server: consente ai programmatori di inviare e ricevere dati da un server attraverso il protocollo HTTP, senza dover scrivere codice complesso.

Axios si basa sulla tecnologia Promise, il che significa che supporta sia le richieste asincrone che quelle sincrone: le richieste asincrone sono particolarmente utili in quanto consentono all'applicazione di continuare a funzionare senza dover attendere la risposta del server; una volta che il server risponde, Axios restituisce i dati in un formato facilmente manipolabile.

Axios offre anche molte funzionalità utili, come la possibilità di impostare le intestazioni delle richieste personalizzate, la gestione degli errori, l'intercettazione delle richieste e delle risposte, e molto altro ancora.

Immaginiamo di voler effettuare una richiesta GET all'URL:

```
https://jsonplaceholder.typicode.com/posts
```

che restituisce un array di oggetti JSON contenenti informazioni sui post di un blog fittizio. La risposta viene poi elaborata all'interno della funzione then, che stampa i dati della risposta sulla console.

Consideriamo la chiamata tramite Fetch API:

```
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

In questo esempio, fetch viene utilizzato per effettuare una richiesta GET all'URL https://jsonplaceholder.typicode.com/posts. La funzione then viene poi utilizzata per convertire la risposta in formato JSON tramite il metodo json(), e successivamente per elaborare i dati della risposta; in caso di errore, la funzione catch viene eseguita, stampando l'errore sulla console.

Ricordiamo che Fetch è una API nativa del browser che può essere utilizzata senza dover installare alcuna libreria esterna, ed è diventata una scelta sempre più popolare per effettuare richieste HTTP asincrone in JavaScript.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Consideriamo ora la stessa chiamata in Ajax jQuery:

```
$.ajax({
  url: 'https://jsonplaceholder.typicode.com/posts',
  method: 'GET',
  success: function(response) {
    console.log(response);
  },
  error: function(error) {
    console.log(error);
  }
});
```

In questo esempio, la funzione $.ajax di jQuery viene utilizzata per effettuare una richiesta GET all'URL https://jsonplaceholder.typicode.com/posts. Quando la richiesta viene completata con successo, la funzione success viene eseguita, stampando la risposta sulla console; in caso di errore, la funzione error viene eseguita, stampando l'errore sulla console.

Per eseguire una richiesta GET verso un server remoto, è possibile utilizzare il seguente codice:

```
const axios = require('axios');
axios.get('https://jsonplaceholder.typicode.com/posts')
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.log(error);
  });
```

In questo esempio, Axios viene utilizzato per effettuare una richiesta GET all'URL https://jsonplaceholder.typicode.com/posts, che restituisce un array di oggetti JSON contenenti informazioni sui post di un blog fittizio. La risposta viene poi elaborata all'interno della funzione then, che stampa i dati della risposta sulla console; in caso di errore, Axios esegue invece la funzione catch, che stampa l'errore sulla console.

Naturalmente, Axios supporta anche altri metodi HTTP come POST, PUT e DELETE, e fornisce molte opzioni per personalizzare le richieste, come ad esempio l'invio di dati JSON o l'impostazione di intestazioni personalizzate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La differenza principale tra Fetch API, Ajax di jQuery ed Axios è la loro sintassi e la flessibilità che offrono.

- La Fetch API è una API nativa del browser che consente di effettuare richieste HTTP asincrone tramite JavaScript. La sintassi è abbastanza semplice e intuitiva, e la Fetch API consente di gestire la risposta in modo flessibile utilizzando le promesse. Tuttavia, la Fetch API può richiedere più codice per gestire le richieste più complesse rispetto ad Axios e jQuery.
- Ajax di jQuery è una libreria JavaScript molto popolare che semplifica notevolmente il processo di effettuare richieste HTTP asincrone tramite JavaScript. La sintassi è semplice e intuitiva, ed è possibile utilizzare molte opzioni per personalizzare le richieste, come ad esempio l'invio di dati JSON o l'impostazione di intestazioni personalizzate. Tuttavia, la libreria può appesantire il codice dell'applicazione e può essere meno flessibile rispetto ad Axios.
- Axios è una libreria JavaScript che semplifica notevolmente il processo di effettuare richieste HTTP asincrone tramite JavaScript. Axios utilizza le Promises per gestire la risposta, il che lo rende molto flessibile e facile da usare. La libreria fornisce anche molte funzionalità utili, come la possibilità di impostare le intestazioni delle richieste personalizzate, la gestione degli errori, l'intercettazione delle richieste e delle risposte, e molto altro ancora. Tuttavia, Axios richiede di essere installato come una libreria esterna e può richiedere un po' di tempo per essere configurato correttamente.

In sintesi, Fetch API, Ajax di jQuery ed Axios sono tutti utilizzati per effettuare richieste HTTP asincrone tramite JavaScript, ma differiscono per la loro sintassi e la flessibilità che offrono: la scelta di quale utilizzare dipenderà dalle esigenze specifiche dell'applicazione e dalle preferenze personali dello sviluppatore.

Ecco un altro esempio di utilizzo di Axios per effettuare una richiesta POST e gestire la risposta:

```
const axios = require('axios');
axios.post('https://jsonplaceholder.typicode.com/posts', {
  title: 'Il titolo del mio post',
  body: 'Il contenuto del mio post',
  userId: 1
})
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.log(error);
  });
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, Axios viene utilizzato per effettuare una richiesta POST all'URL https://jsonplaceholder.typicode.com/posts con un oggetto JSON contenente il titolo, il contenuto e l'ID dell'utente del post; quando la richiesta viene completata con successo, la funzione then viene eseguita, stampando i dati della risposta sulla console; in caso di errore, la funzione catch viene eseguita, stampando l'errore sulla console.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Vuejs

Vue.js è un framework JavaScript open source per la creazione di interfacce utente reattive e dinamiche. Creato da Evan You nel 2014, Vue.js ha guadagnato rapidamente popolarità grazie alla sua semplicità, flessibilità e buone prestazioni.

Il framework si basa sul pattern architetturale **Modello-Vista-ViewModel (MVVM)**, che consente di separare la logica di presentazione dalla logica di business dell'applicazione. Vue.js fornisce un sistema di componenti, che consente di creare interfacce utente modulari e riutilizzabili. Inoltre, il framework utilizza una sintassi dichiarativa e intuitiva per la gestione del rendering dei dati, semplificando notevolmente il processo di sviluppo.

Vue.js supporta anche la programmazione reattiva, che consente di aggiornare automaticamente l'interfaccia utente quando i dati cambiano. Questo rende le applicazioni Vue.js molto veloci ed efficienti, poiché l'interfaccia utente viene aggiornata solo quando necessario.

MVC (Model-View-Controller) e MVVM (Model-View-ViewModel) sono due pattern architetturali utilizzati comunemente nello sviluppo di applicazioni software.

MVC è stato il primo pattern architetturale utilizzato per separare le responsabilità di presentazione, logica di business e gestione dei dati in un'applicazione. In questo pattern, il modello rappresenta la gestione dei dati, la vista rappresenta la presentazione dell'interfaccia utente e il controller gestisce la logica di business dell'applicazione. Il controller funge da intermediario tra la vista e il modello, permettendo alle modifiche della vista di aggiornare il modello e viceversa.

MVVM è una variante di MVC che si concentra sulla separazione tra vista e modello, utilizzando un nuovo componente chiamato ViewModel. In questo pattern, la vista rappresenta ancora l'interfaccia utente, ma la logica di presentazione viene spostata nel ViewModel. Il ViewModel si occupa di mappare i dati dal modello alla vista e viceversa, consentendo alla vista di essere indipendente dalla logica di business dell'applicazione.

In sintesi, la principale differenza tra MVC e MVVM è che MVC utilizza un controller per gestire la logica di business dell'applicazione, mentre MVVM utilizza un ViewModel per gestire la logica di presentazione. Inoltre, MVVM si concentra maggiormente sulla separazione tra vista e modello, consentendo una maggiore flessibilità nell'aggiornamento dell'interfaccia utente. Entrambi i pattern architetturali sono validi e utilizzati comunemente nello sviluppo di applicazioni software, e la scelta tra l'uno e l'altro dipende dalle esigenze specifiche dell'applicazione e dalle preferenze del programmatore.

Analizziamo la differenza tra una implementazione in "standard" MVC ed una in MVVM con Vue.js

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

MVC

Index.js

```
// Controller (index.js)
const express = require('express');
const app = express();
const bodyParser = require('body-parser');
const CounterModel = require('./counterModel');

app.use(bodyParser.urlencoded({ extended: false }));

app.get('/', function (req, res) {
  const model = new CounterModel();
  res.render('view', { counter: model.getCounter() });
});

app.post('/', function (req, res) {
  const model = new CounterModel();

  if (req.body.increment) {
    model.incrementCounter();
  } else if (req.body.reset) {
    model.resetCounter();
  }

  res.render('view', { counter: model.getCounter() });
});

app.listen(3000, function () {
  console.log('App listening on port 3000');
});
```

CounterModel.js

```
// Model (counterModel.js)
class CounterModel {
  constructor() {
    this.counter = 0;
  }

  getCounter() {
    return this.counter;
  }

  incrementCounter() {
    this.counter++;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  }

  resetCounter() {
    this.counter = 0;
  }
}

module.exports = CounterModel;
```

View.ejs

```
<!-- View (view.ejs) -->
<html>
  <head>
    <title>MVC Example</title>
  </head>
  <body>
    <h1>MVC Example</h1>
    <p>Counter: <%= counter %></p>
    <form method="POST">
      <button name="increment">Increment</button>
      <button name="reset">Reset</button>
    </form>
  </body>
</html>
```

MVVM

Index.js

```
<template>
  <div>
    <h1>{{ title }}</h1>
    <p>{{ message }}</p>
    <button @click="incrementCounter()">Incrementa</button>
    <p>Contatore: {{ counter }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      title: 'Benvenuti in Vue.js!',
      message: 'Questo è un esempio di componente Vue.js.',
      counter: 0
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    }
  },
  methods: {
    incrementCounter() {
      this.counter++
    }
  }
}
</script>
```

MVC vs MVVM

In MVVM, viene creato un componente Vue.js che visualizza un titolo, un messaggio, un pulsante e un contatore sulla pagina: la sezione template contiene il markup HTML del componente, mentre la sezione script contiene la logica JavaScript che gestisce il componente.

In MVC, il controller è responsabile di gestire la logica di business dell'applicazione, compreso l'aggiornamento del contatore; in MVVM, invece, il ViewModel è responsabile della logica di presentazione e dell'aggiornamento del contatore.

In questo esempio, Vue.js utilizza MVVM per gestire la logica di presentazione e di aggiornamento del contatore, utilizzando il componente stesso come ViewModel: quando l'utente fa clic sul pulsante, il metodo incrementCounter viene eseguito, aggiornando il valore della variabile counter all'interno del componente; questo a sua volta aggiorna automaticamente la vista, poiché Vue.js utilizza la programmazione reattiva per rilevare le modifiche ai dati e aggiornare l'interfaccia utente di conseguenza.

La scelta tra l'utilizzo di MVC o MVVM dipende dalle esigenze specifiche dell'applicazione e dalle preferenze del programmatore. Entrambi i pattern architetturali offrono vantaggi e svantaggi.

MVC, ad esempio, offre una maggiore separazione tra la logica di presentazione e la logica di business dell'applicazione, semplificando la gestione del codice e migliorando la manutenibilità dell'applicazione. Inoltre, il modello può essere utilizzato anche da altre parti dell'applicazione, rendendo la logica di business dell'applicazione più flessibile e scalabile.

D'altra parte, MVVM offre una maggiore separazione tra la vista e il modello, consentendo una maggiore flessibilità nell'aggiornamento dell'interfaccia utente. Inoltre, il ViewModel permette di mantenere lo stato dell'interfaccia utente, rendendo l'applicazione più reattiva e interattiva.

In generale, se l'applicazione prevede una forte interazione tra la vista e il modello, MVVM potrebbe essere la scelta migliore; d'altra parte, se l'applicazione prevede una maggiore separazione tra la logica di presentazione e la logica di business, MVC potrebbe essere la scelta migliore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Axios & Vuejs

Axios e Vue.js sono due tecnologie importanti per lo sviluppo di applicazioni web moderne e performanti. Axios è una libreria JavaScript utilizzata per effettuare chiamate HTTP e gestire le risposte del server, mentre Vue.js è un framework JavaScript utilizzato per creare interfacce utente reattive e complesse.

Quando utilizzati insieme, Axios e Vue.js consentono di creare applicazioni web potenti ed efficaci. In particolare, Axios fornisce un modo semplice e flessibile per effettuare chiamate API, mentre Vue.js consente di gestire facilmente i dati ricevuti e di renderizzarli in modo dinamico nella pagina.

Grazie alla sua architettura basata sui componenti e alla sua sintassi intuitiva, Vue.js consente di creare facilmente interfacce utente complesse e reattive. Inoltre, Vue.js è altamente personalizzabile e può essere integrato facilmente con altre librerie e framework, tra cui Axios.

Ecco un esempio di come utilizzare Axios con Vue.js per effettuare una chiamata API e visualizzare i dati ottenuti nella pagina:

```
<template>
  <div>
    <h1>Lista post</h1>
    <ul>
      <li v-for="post in posts" :key="post.id">{{ post.title }}</li>
    </ul>
  </div>
</template>

<script>
import axios from 'axios';

export default {
  data() {
    return {
      posts: []
    }
  },
  mounted() {
    axios.get('https://jsonplaceholder.typicode.com/posts')
      .then(response => {
        this.posts = response.data;
      })
      .catch(error => {
        console.log(error);
      });
  }
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
</script>
```

In questo esempio, stiamo utilizzando Axios per effettuare una chiamata HTTP GET all'endpoint https://jsonplaceholder.typicode.com/posts, che restituisce una lista di post in formato JSON. Una volta ricevuti i dati, li assegniamo alla proprietà posts dell'oggetto Vue.js, che a sua volta viene utilizzata per visualizzare i titoli dei post nella pagina.

In questo codice, la parte di Axios è quella relativa alla chiamata HTTP effettuata attraverso il metodo axios.get(). Questo metodo è fornito dalla libreria Axios e viene utilizzato per effettuare richieste HTTP.

La parte di Vue.js è invece quella relativa alla gestione dei dati e alla visualizzazione nella pagina. In particolare, la proprietà posts viene utilizzata per memorizzare i dati ricevuti dalla chiamata API e il ciclo v-for viene utilizzato per iterare su questi dati e visualizzarli nella pagina.

Il metodo mounted() è un metodo del ciclo di vita di un componente in Vue.js, che viene chiamato automaticamente quando il componente viene montato nel DOM. In particolare, il metodo mounted() viene chiamato dopo che il componente è stato inizializzato e il suo template è stato renderizzato nella pagina. Tale metodo è spesso utilizzato per effettuare azioni o inizializzare dati quando il componente viene visualizzato nella pagina: ad esempio, è comune utilizzarlo per effettuare chiamate API e ottenere dati dal server, in modo che i dati siano disponibili quando il componente viene visualizzato.

In pratica, all'interno del metodo mounted() si possono definire le istruzioni da eseguire quando il componente viene montato nel DOM: ad esempio, si possono effettuare chiamate API tramite Axios per ottenere i dati necessari, oppure si possono inizializzare variabili e proprietà del componente.

È importante notare che tale metodo viene chiamato solo una volta, quando il componente viene montato nel DOM: se si desidera aggiornare i dati del componente in risposta a un evento o a una modifica di stato, è necessario utilizzare altri metodi del ciclo di vita di Vue.js, come ad esempio il metodo updated().

In generale, Axios viene utilizzato per effettuare chiamate HTTP e ottenere dati dal server, mentre Vue.js viene utilizzato per gestire questi dati e renderizzarli nella pagina in modo reattivo. Le due tecnologie possono essere utilizzate insieme per creare applicazioni web moderne e performanti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).